Ryo Leo Iwata

CollabBoard Architecture Plan

Executive Summary

This architecture prioritizes rapid MVP delivery (24-hour target) using Firebase's managed real-time infrastructure, React/Konva for declarative canvas rendering, and Anthropic Claude for AI board manipulation. The stack trades long-term flexibility and cost optimization for immediate development velocity and minimal infrastructure management.

Architecture Overview

System Components

Frontend Stack

- Next.js 14+ with App Router for SSR/SSG capabilities
- React 18 with Konva.js (react-konva) for canvas rendering
- Zustand for lightweight state management
- TailwindCSS for UI components outside canvas

Backend Stack

- Firebase Firestore for persistent board object storage
- Firebase Realtime Database for ephemeral cursor/presence data
- Firebase Auth for user authentication
- Firebase Hosting or Vercel for deployment

AI Layer

- Anthropic Claude Sonnet 4.5 for natural language command processing
- Vercel AI SDK for streaming and tool orchestration
- Direct function calling pattern for board manipulation

Detailed Trade-off Analysis

Database Architecture

Decision: Dual-database pattern (Firestore + Realtime DB)

Firestore for Board Objects

- One document per board object (sticky notes, shapes, frames)
- Collection structure: `boards/{boardId}/objects/{objectId}`
- Enables real-time listeners on object creation/updates/deletion
- Query capabilities for filtering by type, color, position

Realtime DB for Ephemeral State

- Cursor positions (30-60 Hz update rate)
- User presence indicators
- Typing indicators
- Path structure: `/sessions/{boardId}/users/{userId}/cursor`

Strengths:

- Zero custom WebSocket server implementation needed
- Automatic conflict resolution via Firebase's operational transforms
- Real-time listeners provide instant sync across clients
- Presence detection is a documented pattern with automatic cleanup
- Security rules enforce authorization at the database level

Weaknesses:

- Firestore 1 write/second per document limit requires object-level documents (can't store entire board state in one doc)
- Realtime DB costs scale with connection-hours and bandwidth (cursor updates at 60 Hz × N users gets expensive)
- Vendor lock-in makes migration difficult (Firebase-specific SDK patterns throughout codebase)
- Cold start costs for Firestore queries (~100-200ms for first read)
- Complex queries require pre-defined composite indexes (24-48 hour index build time)

Mitigations:

- Throttle cursor updates to 30 Hz (33ms intervals) to reduce Realtime DB bandwidth
- Implement viewport culling—only sync objects visible in current pan/zoom bounds
- Use Firestore's `onSnapshot` with `includeMetadataChanges: false` to avoid double-renders
- Cache board metadata (title, collaborators) in localStorage to reduce reads

Cost Implications:

- Firestore: $0.18/100K reads, $0.18/100K writes, $0.02/GB storage
- Realtime DB: $5/GB downloaded, $1/GB stored, $5/GB downloaded
- For 1,000 active users with 10-minute sessions: ~$150-300/month in cursor sync alone

Canvas Rendering

Decision: Konva.js with react-konva wrapper

Strengths:

- Declarative React API maps directly to Zustand state (`<Rect fill={color} />`)
- Built-in hit detection, drag-and-drop, and transform controls
- Layer system supports z-index management without manual sorting
- Event delegation handles onClick/onDrag at the shape level
- Caching API (`cache()`) for static objects improves render performance

Weaknesses:

- Performance degrades beyond 1,000 objects without manual optimization
- No built-in viewport culling (you must filter visible objects before render)
- React reconciliation overhead on every state change (even off-screen objects re-render)
- TypeScript types lag behind Konva API updates
- Limited support for advanced graphics (shaders, filters) compared to PixiJS

Performance Optimization Strategy:

Viewport Culling: Filter objects outside current view bounds before passing to Konva

```
 const visibleObjects = objects.filter(obj =>
  isInViewport(obj, viewport.x, viewport.y, viewport.width, viewport.height)
)
```

1.

Layer Splitting: Separate static (background grid) from dynamic (shapes) layers

```
 <Layer listening={false}>{/* Static grid */}</Layer>
<Layer>{/* Interactive shapes */}</Layer>
```

2.
3. **Debounced Sync**: Only write to Firestore on drag end, not during drag (local optimistic updates)

4. **Object Pooling**: Reuse Konva shape instances for repeated object types

Target Performance:

- 60 FPS during pan/zoom with <500 objects
- <100ms sync latency for object creation

- <50ms cursor position updates

Alternative Considered: HTML5 Canvas + manual rendering loop (lower-level control, steeper learning curve, less AI-friendly code generation)

Real-Time Synchronization

Decision: Firestore listeners with optimistic updates

Data Flow:

1. User action (create sticky note) → Update local Zustand store immediately
2. Write to Firestore → `updateDoc(docRef, data)`
3. Firestore listener triggers → `onSnapshot(collection, callback)`
4. Callback updates Zustand store → React re-renders Konva components

Conflict Resolution:

- **Last-write-wins** for object properties (position, color, text)
- Firestore's server timestamp ensures consistent ordering
- No custom CRDTs or operational transform logic required

Strengths:

- Firestore handles all conflict resolution server-side
- `onSnapshot` provides exactly-once delivery guarantees
- Offline persistence built-in (local cache continues working)
- Security rules prevent unauthorized writes (`allow update: if request.auth != null`)

Weaknesses:

- Last-write-wins can result in lost edits during simultaneous typing
- No granular field-level merging (entire document overwrites)
- Network latency creates perceived lag (50-200ms depending on region)
- Firestore emulator required for local development (adds setup friction)

Simultaneous Edit Handling:

- **Accepted risk**: If two users edit the same sticky note text simultaneously, last write wins
- **Mitigation**: Show "User X is editing" indicator using Realtime DB presence
- **Not implemented**: Field-level locking or merge conflicts (out of scope for MVP)

Testing Plan:

1. Two browser windows, different users, create 20 sticky notes simultaneously

2. Network throttling (Fast 3G) to verify sync works under poor conditions
3. Kill network mid-edit, verify Firestore offline cache persists changes
4. Refresh page during multi-user session, verify state reloads correctly

Authentication

Decision: Firebase Auth with Google/GitHub providers

Strengths:

- Pre-built FirebaseUI component gets auth working in <30 minutes
- Built-in session management (JWT refresh, secure cookie handling)
- Integrates directly with Firestore security rules
- Supports anonymous auth for "guest mode" if needed

Weaknesses:

- Requires OAuth app setup for social providers (adds deployment steps)
- FirebaseUI has limited customization (branded login screens require custom implementation)
- No built-in multi-tenancy (if you need workspace-level isolation later)

Security Rules Example:

```
match /boards/{boardId}/objects/{objectId} {
  allow read: if isAuthenticated();
  allow write: if isAuthenticated() && isCollaborator(boardId);
}
```

AI Agent Architecture

Decision: Claude Sonnet 4.5 with Vercel AI SDK

Tool Schema Design:

- Define board manipulation functions as Claude tools
- Each tool maps to a Firestore write operation
- Tools return success/failure for Claude to confirm completion

Tool Examples:

```
const tools = {
  createStickyNote: {
    description: 'Create a sticky note at specified position',
    parameters: z.object({
```

```
      text: z.string(),
      x: z.number(),
      y: z.number(),
      color: z.enum(['yellow', 'pink', 'blue', 'green'])
    }),
    execute: async ({ text, x, y, color }) => {
      const docRef = await addDoc(collection(db, `boards/${boardId}/objects`), {
        type: 'stickyNote',
        text, x, y, color,
        createdAt: serverTimestamp()
      })
      return { success: true, id: docRef.id }
    }
  }
}
```

Strengths:

- Claude Sonnet 4.5 has strong function-calling reliability (95%+ accuracy in testing)
- Vercel AI SDK handles streaming, tool call parsing, and retries
- Multi-step commands work via sequential tool calls in one completion
- Cost per command: $0.003-0.01 depending on board state context

Weaknesses:

- Board state context grows linearly with object count (500 objects = ~50K tokens input)
- Token costs scale badly beyond 1,000 objects on board
- No built-in memory of previous commands (stateless)
- Complex spatial reasoning ("arrange evenly") requires explicit grid math in tools
- Vercel AI SDK is tightly coupled to Next.js route handlers (not portable)

Cost Optimization Strategies:

1. **Context Pruning**: Only include objects relevant to command (e.g., only pink sticky notes for "move all pink notes")
2. **Summarized State**: Send object count + bounding box instead of full object details
3. **Command History**: Store last 5 commands in prompt to improve multi-turn understanding
4. **Batch Operations**: Single tool call for "create 10 sticky notes" instead of 10 separate calls

AI Cost Projections:

- **Development**: ~$10-20 for testing (200-400 AI commands during build)
- **100 users**: 5 commands/user/month × $0.005/command = ~$2.50/month

- **1,000 users**: ~$25/month
- **10,000 users**: ~$250/month (need context pruning by this scale)
- **100,000 users**: ~$2,500/month (requires caching layer and command batching)

Deployment

Decision: Vercel (frontend + API routes) + Firebase (auth/database/hosting fallback)

Strengths:

- Git push deploys with zero config
- Edge runtime for API routes (low latency globally)
- Built-in preview deployments for PRs
- Vercel AI SDK native integration

Weaknesses:

- Vercel's free tier limits (100GB bandwidth, 100 serverless function invocations/day)
- Cold starts on serverless functions (~200-500ms first request)
- No WebSocket support (must use Firestore for real-time)

Alternative Considered: Firebase Hosting + Cloud Functions (simpler but slower function cold starts, no edge deployment)

Risk Assessment

Critical Risks

1. Firestore Write Limit (1 write/sec per document)

- **Impact**: High-frequency cursor updates can't use Firestore
- **Mitigation**: Use Realtime DB for cursors, Firestore only for persistent objects
- **Residual Risk**: Low (well-documented pattern)

2. Cursor Sync Bandwidth Costs

- **Impact**: 30 Hz cursor updates × 10 concurrent users × 10 min session = significant Realtime DB bandwidth
- **Mitigation**: Throttle to 20 Hz, implement distance-based updates (only send if cursor moved >5px)
- **Residual Risk**: Medium (requires monitoring at scale)

3. AI Token Costs with Large Boards

- **Impact**: 1,000+ object board = 100K+ input tokens per AI command

- **Mitigation**: Context pruning, implement "active region" focusing
- **Residual Risk**: High (requires architectural change at scale)

4. Konva Performance Degradation

- **Impact**: 500+ objects causes frame drops during pan/zoom
- **Mitigation**: Viewport culling, layer splitting, object caching
- **Residual Risk**: Medium (requires ongoing optimization)

Scalability Constraints

Firestore Limitations:

- 1 million concurrent connections (not a concern for this project)
- 10,000 writes/second per database (shared across all collections)
- 1MB document size limit (single sticky note won't hit this)

Realtime DB Limitations:

- 200,000 concurrent connections per database
- 1,000 writes/second per node
- No querying (key-value only)

Breaking Points:

- **10,000 concurrent users**: Realtime DB bandwidth costs become prohibitive ($1,000+/month)
- **5,000 objects per board**: Konva rendering falls below 30 FPS without aggressive optimization
- **50,000 AI commands/day**: Need caching layer to avoid $500+/month token costs

Alternative Architectures Considered

Option B: Custom WebSocket Server

Stack: Node.js + Socket.io + PostgreSQL + Redis

- **Pros:** Full control over sync protocol, lower per-connection costs, better performance at scale
- **Cons:** 40+ hours to implement reliable conflict resolution, presence tracking, and reconnection logic. High operational burden.

Option C: Supabase (Postgres + Realtime)

Stack: Supabase Realtime + Postgres + PostgREST

- **Pros:** Open-source alternative to Firebase, SQL querying, row-level security
- **Cons:** Realtime engine less mature than Firebase, requires more manual setup for presence/cursors

Option D: Yjs CRDT + WebRTC

Stack: Yjs + y-websocket + simple signaling server

- **Pros:** True peer-to-peer sync, no backend costs for sync, automatic conflict resolution
- **Cons:** Complex NAT traversal, TURN server costs, steep learning curve, poor AI tool support

Why Firebase Won:

- Time-to-MVP is 3-5x faster than custom WebSocket server
- Firebase Realtime DB patterns for cursors/presence are well-documented
- Security rules eliminate need for custom API authorization layer
- AI coding tools (Claude Code, Cursor) generate more reliable Firebase code than custom server code

Implementation Roadmap

Phase 1: Cursor Sync (Hours 0-4)

1. Set up Next.js + Firebase project
2. Implement Firebase Auth with Google provider
3. Create Realtime DB schema for cursor positions
4. Build cursor component with Konva
5. Test two-browser cursor sync

Gate: Two users see each other's cursors moving in real-time

Phase 2: Object Sync (Hours 4-12)

1. Design Firestore schema (boards/{boardId}/objects/{objectId})
2. Implement sticky note creation with Zustand store
3. Set up Firestore listeners for real-time object sync
4. Add drag-and-drop with optimistic updates
5. Test simultaneous sticky note creation

Gate: Two users can create and move sticky notes simultaneously

Phase 3: Board Features (Hours 12-18)

1. Add shape primitives (rectangle, circle, line)
2. Implement text editing with contentEditable
3. Build selection system (single + multi-select)
4. Add color picker and delete operations

Gate: Full CRUD operations work for all object types

Phase 4: AI Integration (Hours 18-22)

1. Set up Vercel AI SDK with Claude Sonnet 4.5
2. Define tool schema for board manipulation
3. Implement `getBoardState()` context provider
4. Build chat UI for AI commands
5. Test 6+ command types

Gate: AI can execute creation, manipulation, and layout commands

Phase 5: Polish (Hours 22-24)

1. Add loading states and error handling
2. Implement presence indicators
3. Deploy to Vercel
4. End-to-end testing with 5 concurrent users

Success Criteria

MVP Pass Requirements:

- [ ] Two users can see each other's cursors and edits in <100ms
- [ ] Board survives refresh with all objects persisted
- [ ] AI agent executes 6+ distinct command types
- [ ] 5 concurrent users without frame drops
- [ ] Deployed and publicly accessible with auth

Performance Targets:

- 60 FPS during pan/zoom with 200 objects
- <100ms object sync latency
- <50ms cursor sync latency
- <2s AI command execution (simple commands)

Conclusion

This architecture optimizes for the 24-hour MVP deadline by leveraging Firebase's managed real-time infrastructure and Konva's declarative canvas API. The primary trade-offs are vendor lock-in and scaling costs in exchange for development velocity. For a one-week sprint where the goal is learning AI-first development workflows, these trade-offs are acceptable. Post-MVP, the architecture would require significant refactoring for production scale (10,000+ concurrent users), including a custom sync protocol, viewport-based object loading, and AI response caching.

CollabBoard Pre-Search Checklist

Here's the full Pre-Search checklist answered against your chosen stack.

Phase 1: Define Your Constraints

1. Scale & Load Profile

Users at launch: 5-10 (you, reviewers, and testers during the sprint). In 6 months: irrelevant — this is a sprint project with a Sunday deadline, not a product launch. Design for the grading criteria (5+ concurrent users without degradation) and nothing beyond that.

Traffic pattern: Spiky. Zero traffic most of the time, then bursts when reviewers test simultaneously. The evaluation explicitly says they'll test with 5+ concurrent users, so your spike target is clear and small.

Real-time requirements: Yes, this is the core of the project. Cursor sync at <50ms requires persistent connections (Firebase Realtime DB handles this via WebSockets under the hood). Object sync at <100ms uses Firestore's `onSnapshot` listeners, also WebSocket-based. You never manage WebSocket connections directly — Firebase abstracts that entirely.

Cold start tolerance: Moderate. Firebase Hosting and Firestore have no cold start. If you use Firebase Cloud Functions for AI orchestration, cold starts are 1-3 seconds on the free tier — problematic for the <2 second AI response target. Mitigation: use a Vercel API route (Next.js serverless function) instead, which has faster cold starts (~200-500ms), or keep a Cloud Function warm with a scheduled ping. Better yet, since your AI commands aren't latency-critical at the cold-start level (users expect a brief wait when talking to an AI), a 1-second cold start plus 1-second LLM response is acceptable.

2. Budget & Cost Ceiling

Monthly spend limit: Effectively $0-25 during the sprint. Firebase's Spark (free) plan gives you 100 simultaneous connections, 1GB Realtime DB storage, 1GB Firestore storage, 50K Firestore reads/day, and 20K writes/day. That's more than enough for development and evaluation. The Anthropic API for Claude Sonnet costs roughly $3 per million input tokens and $15 per million output tokens — during development you'll spend maybe $2-5 total on AI commands.

Pay-per-use acceptable? Yes, ideal. Firebase Blaze (pay-as-you-go) and Anthropic's API are both usage-based, meaning you pay nothing when nobody's using the app. For a sprint project with intermittent usage, this is vastly cheaper than any fixed-cost plan.

Where will you trade money for time? Everywhere possible. Firebase's managed real-time sync costs more per operation than a self-hosted WebSocket server, but saves you 10-15 hours of infrastructure code. Vercel's free tier saves you from configuring CI/CD. Claude Sonnet costs more per token than running an open-source model, but eliminates GPU provisioning and model serving entirely. Every dollar spent on managed services buys you hours to focus on features that get graded.

3. Time to Ship

MVP timeline: 24 hours. This is a hard gate — auth, deployment, real-time sync, cursors, sticky notes, one shape type, all working in production.

Speed-to-market vs. long-term maintainability: Speed wins overwhelmingly. The project ends Sunday. Maintainability matters only insofar as you can debug and iterate during the week. Code cleanliness helps you move fast on days 3-7, but architectural elegance that takes extra hours to set up is a net negative.

Iteration cadence after launch: Daily. You ship the MVP Tuesday, then have until Friday for full features and Sunday for polish. Expect to deploy 5-10 times per day. Vercel's git-push deploys make this painless.

4. Compliance & Regulatory Needs

HIPAA / GDPR / SOC 2 / Data residency: None apply. This is an educational sprint project, not a production SaaS. Firebase stores data in Google Cloud's us-central1 by default, which is fine. You're not handling health data, you likely won't have EU users during evaluation, and there's no enterprise client requirement.

That said, Firebase Auth handles user credentials securely by default (passwords are hashed, tokens are JWTs), and Firestore security rules prevent unauthorized data access. You get baseline security without effort, which is enough.

5. Team & Skill Constraints

Solo or team: Assume solo (the PRD reads as an individual assignment). This is the most important constraint in the entire checklist — it means every technology choice must optimize for "one person can build, debug, and deploy this in a week."

Languages/frameworks you know well: This stack assumes strong JavaScript/TypeScript and React familiarity. If that's accurate, React + Next.js + Zustand will feel natural. If you're more comfortable with Python, Vue, or another ecosystem, that changes the calculus significantly — but the PRD's emphasis on AI-first development with Claude Code and Cursor (both heavily optimized for React/TypeScript) pushes toward this stack regardless.

Learning appetite vs. shipping speed: Shipping speed. The only new technologies in this stack you might need to learn are react-konva (for canvas rendering) and the Vercel AI SDK (for LLM tool calling). Both have small API surfaces and good documentation. Budget 1-2 hours of learning each, then build as you learn. Don't pre-study — write real project code from minute one.

Phase 2: Architecture Discovery

6. Hosting & Deployment

Serverless vs. containers vs. edge vs. VPS: Serverless. Vercel deploys your Next.js app as serverless functions (API routes) and static/edge-rendered pages. Firebase provides serverless everything — Firestore, Realtime DB, Auth, and Hosting are all fully managed. You never provision a server, container, or VM. This eliminates an entire category of work (scaling, load balancing, health checks, process management) that would consume hours in a sprint.

CI/CD requirements: Minimal. Vercel auto-deploys from your GitHub main branch on every push. No pipeline configuration needed. If you want preview deployments for pull requests, Vercel does that automatically too. Firebase Hosting can deploy via `firebase deploy` from CLI or a GitHub Action, but Vercel is simpler for the Next.js frontend.

Scaling characteristics: Firebase and Vercel both auto-scale. Firestore handles up to 10,000 writes/second per database and 1 million concurrent connections. Realtime DB handles 200,000 simultaneous connections on the Blaze plan. Vercel's serverless functions scale to demand. None of these limits are remotely relevant at 5-10 users, but they mean you never have to think about scaling during the sprint.

7. Authentication & Authorization

Auth approach: Google sign-in as the primary method — it's the fastest to implement with Firebase Auth, requires zero email verification flow, and every evaluator has a Google account.

Add email/password as a fallback for evaluators who don't want to use Google. Skip magic links (adds email delivery complexity) and SSO (enterprise feature, not needed).

RBAC needed? No. All authenticated users have equal access to a board. If you want to scope boards to their creators, a simple Firestore security rule (`allow read, write: if request.auth != null && resource.data.members[request.auth.uid] == true`) handles it without any role system.

Multi-tenancy considerations: Each board is its own "tenant" — Firestore documents under `boards/{boardId}/objects/*` are naturally scoped. Users can create multiple boards. No shared infrastructure concerns.

8. Database & Data Layer

Database type: Document (Firestore) for persistent board state, plus a real-time key-value store (Firebase Realtime DB) for ephemeral data. This dual-database pattern is the core of your architecture.

Firestore stores board objects as documents: each sticky note, shape, frame, or connector is a document in `boards/{boardId}/objects/{objectId}` with fields like `{type, x, y, width, height, color, text, zIndex, createdBy, updatedAt}`. Firestore's `onSnapshot` provides real-time sync for all connected clients.

Realtime DB stores cursor positions and presence: `/rooms/{boardId}/cursors/{userId}` holds `{x, y, color, displayName}` and gets overwritten 15-20 times per second. `/rooms/{boardId}/presence/{userId}` uses `.info/connected` and `onDisconnect` to automatically clean up when a user leaves.

Why two databases? Firestore charges per document read/write and is optimized for structured queries on persistent data. Realtime DB charges per bandwidth and is optimized for high-frequency overwrites of small payloads. Cursor positions are overwritten constantly and never queried historically — Realtime DB is 5-10x cheaper and faster for this pattern. Board objects are created once, updated occasionally, and need querying (e.g., "get all objects for this board") — Firestore is the right tool.

Read/write ratio: For board objects, heavily read-biased (many clients reading, occasional writes). For cursors, 1:1 (every user writes their own position, reads everyone else's). For presence, write-once-read-many.

9. Backend/API Architecture

Monolith or microservices: Monolith. Your entire backend is one Next.js app with a few API routes. Firebase services are accessed via client SDKs (no backend server needed for sync)

and the AI orchestration is one or two serverless API routes. There's zero reason to split this into services for a sprint project.

REST vs. GraphQL vs. tRPC vs. gRPC: REST, by default — your Next.js API routes are REST endpoints. But honestly, you only have 1-2 API routes total (the AI command endpoint and maybe a board creation endpoint). Everything else goes through Firebase client SDKs directly. Don't overthink the API design.

Background job and queue requirements: None. All operations are synchronous request/response. AI commands are called, executed, and return within 2 seconds. If you later needed long-running AI workflows (e.g., "analyze this entire board and reorganize it"), you'd add a queue, but that's not in the MVP scope.

10. Frontend Framework & Rendering

SEO requirements: None. A collaborative whiteboard is a fully authenticated, interactive app. No search engine needs to crawl it. This means you don't need Next.js's SSR — you could use a pure SPA with Vite. However, Next.js gives you API routes (used for AI orchestration) and Vercel's deployment integration, which are worth the slight overhead.

Offline support/PWA: Not required by the PRD. A collaborative whiteboard without network connectivity is just a local drawing app. Skip service workers and offline caching entirely. If a user disconnects, your Firestore listeners will automatically reconnect and re-sync when the network returns — that's the "graceful disconnect/reconnect handling" the PRD requires, and Firebase provides it for free.

SPA vs. SSR vs. static vs. hybrid: SPA with a thin server layer. The whiteboard canvas is entirely client-rendered (Konva draws to `<canvas>`, no DOM elements for SSR to help with). Your Next.js app effectively runs as a SPA with API routes for the AI backend. Configure Next.js pages to use `'use client'` at the top to avoid SSR for the board component.

11. Third-Party Integrations

External services needed:

The Anthropic Claude API is your only non-Firebase external service. Rate limit for Claude Sonnet is 4,000 requests per minute on the default tier — irrelevant at your scale. Cost per command is $0.003-0.01 as estimated. No pricing cliff until you're doing thousands of commands per day.

Firebase services (Firestore, Realtime DB, Auth, Hosting) are all within one ecosystem. No integration friction between them.

Vercel for hosting. Free tier gives you 100GB bandwidth/month, serverless function invocations, and automatic deployments. No pricing cliff for a sprint project.

Vendor lock-in risk: High for Firebase, low practical impact. Your Firestore data model, security rules, and Realtime DB patterns are Firebase-specific. Migrating to Supabase or a custom backend later would require rewriting your entire sync layer. For a one-week sprint, this doesn't matter. If this were a startup, you'd want to abstract the data layer behind an interface — but that abstraction costs hours you don't have.

Phase 3: Post-Stack Refinement

12. Security Vulnerabilities

Known pitfalls for your stack:

Firestore security rules are your entire authorization layer, and misconfigured rules are Firebase's number one security issue. The most common mistake is leaving rules in test mode (`allow read, write: if true`) and forgetting to lock them down. Write your rules on day one: require authentication for all reads/writes, scope object access to board members, and validate that writes conform to your data schema (e.g., `type` must be one of `['sticky', 'shape', 'frame', 'connector']`).

API route exposure: your AI orchestration endpoint (`/api/ai-command`) must validate the Firebase Auth token on every request. Without this, anyone can send AI commands against any board. Use `firebase-admin` to verify the ID token server-side.

Client-side API keys: Firebase client config (apiKey, authDomain, etc.) is intentionally public — security is enforced by Firestore rules and Auth, not by hiding the config. But your Anthropic API key must never be in client-side code. It lives in Vercel environment variables and is only accessed in server-side API routes.

Dependency risks: `react-konva` and `konva` are maintained by a single developer (Anton Lavrenov). The library is stable and widely used, but if a critical bug appears mid-sprint, you won't get an upstream fix quickly. Mitigate by pinning your dependency versions in `package.json` and not upgrading mid-sprint.

13. File Structure & Project Organization

Monorepo, single Next.js project. Here's the structure:

```
collabboard/
├── app/                    # Next.js App Router
│   ├── page.tsx            # Landing/login page
│   ├── board/[id]/page.tsx   # Board view (main canvas)
```

```
│   └── api/
│       └── ai-command/route.ts  # AI orchestration endpoint
├── components/
│   ├── canvas/              # Konva canvas components
│   │   ├── Board.tsx        # Main Stage + Layer
│   │   ├── StickyNote.tsx   # Sticky note Konva group
│   │   ├── Shape.tsx        # Rectangle/circle Konva shapes
│   │   ├── Connector.tsx    # Line/arrow between objects
│   │   ├── Frame.tsx        # Frame grouping component
│   │   ├── RemoteCursor.tsx # Other users' cursor display
│   │   └── SelectionRect.tsx # Drag-to-select overlay
│   ├── toolbar/             # UI toolbar components
│   └── ui/                  # Shared UI (auth, presence list)
├── stores/
│   ├── boardStore.ts        # Zustand: board objects state
│   ├── cursorStore.ts       # Zustand: cursor positions
│   └── presenceStore.ts     # Zustand: who's online
├── lib/
│   ├── firebase.ts          # Firebase client init
│   ├── firebaseAdmin.ts     # Firebase Admin SDK (server only)
│   ├── sync.ts              # Firestore/RTDB listener setup
│   └── ai/
│       ├── tools.ts         # Tool definitions for Claude
│       ├── executor.ts      # Tool call → Firestore write mapping
│       └── context.ts       # Board state serialization for prompts
├── types/
│   └── board.ts             # TypeScript types for board objects
└── firebase/
    ├── firestore.rules      # Security rules
    └── database.rules.json  # Realtime DB rules
```

The key organizational principle is separation by concern, not by feature. Canvas components are together because they share Konva as a dependency. Stores are together because they share Zustand patterns. The AI layer is isolated in `lib/ai/` so it can be developed and tested independently of the canvas.

14. Naming Conventions & Code Style

Naming patterns: PascalCase for React components (`StickyNote.tsx`), camelCase for functions and variables (`createStickyNote`), camelCase for Zustand store files (`boardStore.ts`), kebab-case for API routes (`ai-command/`). Firestore collection names in camelCase (`boardObjects`). TypeScript interfaces with descriptive names, no `I` prefix (`BoardObject`, not `IBoardObject`).

Linter and formatter: ESLint with Next.js's default config (`eslint-config-next`) plus Prettier for formatting. Set these up in minute one — AI coding tools generate inconsistently formatted code, and auto-formatting on save keeps the codebase readable. Add `"editor.formatOnSave": true` to your VS Code settings.

Configuration: `tsconfig.json` with `strict: true` (catches bugs that would waste debugging time later in the sprint). Prettier with `singleQuote: true, semi: true, trailingComma: 'es5'` — the default for most Next.js projects.

15. Testing Strategy

Unit, integration, e2e tools: For this sprint, manual testing with multiple browser windows is your primary testing strategy. The PRD's testing scenarios (2 users editing simultaneously, refresh mid-edit, rapid creation, network throttling, 5+ concurrent users) are all manual integration tests that you should run daily.

If you have time (likely day 5-7), add lightweight automated tests: Vitest for unit testing AI tool execution logic (does `executor.ts` correctly map a `createStickyNote` tool call to the right Firestore document?), and optionally Playwright for an e2e smoke test (log in, create a sticky note, verify it appears).

Coverage target for MVP: 0%. This is a one-week sprint. Write tests only for code that keeps breaking during manual testing — that's where automated tests have the highest ROI. Don't aim for a coverage number.

Mocking patterns: If you do write tests, mock Firebase with `firebase/testing` or a simple in-memory object store. Mock the Anthropic API by returning hardcoded tool call responses. Never call real external services in tests.

16. Recommended Tooling & DX

VS Code extensions:

The Firebase extension (`firebase.vscode-firebase`) gives you Firestore rules validation and emulator integration. ES7+ React/Redux/React-Native snippets for faster component creation. Prettier and ESLint extensions for auto-formatting. Tailwind CSS IntelliSense if you're using Tailwind for non-canvas UI. GitLens for tracking changes during rapid iteration.

CLI tools: Firebase CLI (`firebase-tools`) for deploying rules, running the local emulator suite (useful for testing Firestore rules without hitting production), and managing the project. Vercel CLI for manual deployments and environment variable management. `npx create-next-app@latest` to scaffold the project in under a minute.

Debugging setup: Chrome DevTools is your primary debugger. The Network tab is critical — use it to watch Firestore and Realtime DB WebSocket frames to verify sync is working. The Performance tab helps you identify canvas rendering bottlenecks (look for long frames during pan/zoom). React DevTools shows you which components re-render on state changes — essential for finding the Zustand/Konva re-render issues mentioned in the cons section. The Firebase Emulator Suite lets you inspect Firestore reads/writes locally without cost, but setting it up takes 30-60 minutes — worth it if you're debugging security rules, skip it otherwise.

AI Conversation

Here's a summary of the questions you asked and the progression of your decision-making process throughout this conversation:

Question 1: Comprehensive Technical Architecture Design You started by uploading the CollabBoard PRD and asked for a full technical architecture analysis. You wanted constraint identification, two stack options (high-speed vs. high-control), trade-off analysis across DX/cost/performance, and a vertical slice implementation plan to hit the 24-hour MVP deadline. This established the two candidate architectures and the recommendation to go with Option A (Firebase + React/Konva + Zustand + Claude).

Question 2: Firebase vs. Liveblocks — Which Real-Time Backend? You shared the Ably comparison article between Firebase and Liveblocks and asked how to navigate the choice between them. This was a targeted deep-dive into the real-time infrastructure decision. The answer walked through pricing, feature coverage, presence limits, collaboration capabilities, and concluded that Firebase was the better fit because Liveblocks' pre-built collaboration features don't help when you're building a custom canvas, and Firebase bundles more of the stack (auth, hosting, serverless functions) in one platform.

Question 3: Frontend Canvas Library — Konva vs. tldraw vs. PixiJS With the Firebase backend locked in, you asked for pros and cons of each frontend option specifically in the context of that backend. This was about understanding how the frontend choice interacts with Firestore's data model and sync pipeline. The answer compared react-konva's transparent data flow, tldraw's sync adapter complexity, and PixiJS's missing interaction layer, concluding that Konva + Zustand gives the cleanest integration with Firebase for a one-week sprint.

Question 4: AI Orchestration — Direct Function Calling vs. LangGraph With both backend and frontend decided, you narrowed focus to the AI integration layer. You asked for pros and cons of a thin orchestrator with direct function calling versus a workflow engine like LangGraph, given your Firebase + Konva stack. The answer explained how direct function calling leverages the existing Firestore sync pipeline (AI writes to Firestore, all clients get updates automatically),

while LangGraph adds infrastructure complexity and latency that doesn't pay off within a one-week timeline.

Question 5: Full Pre-Search Checklist You asked for all 16 sections of the PRD's Pre-Search Checklist answered against your finalized stack. This covered scale and load profiles, budget, time to ship, compliance, team constraints, hosting, auth, database design, API architecture, frontend rendering, third-party integrations, security vulnerabilities, file structure, naming conventions, testing strategy, and recommended tooling. This was your most comprehensive question — essentially documenting every architectural decision and its justification in the format the PRD requires for submission.

Question 6: Architecture Documentation as a Word Document You asked for a formal architecture document with trade-offs, which you asked twice (the first attempt was interrupted). This was about producing a polished deliverable from all the decisions made in the previous questions.

The Overall Arc

Your questions followed a logical narrowing pattern: you started broad (full architecture with two options), then drilled into each layer one at a time (backend real-time choice, then frontend canvas choice, then AI orchestration choice), then expanded back out to document everything comprehensively (pre-search checklist), and finally asked for a formal deliverable. Each question built on the decisions locked in by the previous one, which meant by the time you reached the pre-search checklist, every answer was grounded in a specific, justified stack rather than abstract comparisons.